

Linux mm 初学者学习文档

基于仓库内源码树 Linux 7.0.0-rc7 整理

生成日期：2026-04-12

目录

1 文档定位	2
2 整体路线总表	2
3 第 0 块：先建立脑图，不要急着看所有子系统	3
4 第 1 块：物理内存、页分配与分配失败后的联动	4
5 第 2 块：进程地址空间、VMA、页表与缺页异常	5
6 第 3 块：页缓存、文件映射与文件页 fault	7
7 第 4 块：回收、反向映射、交换与 OOM	8
8 第 5 块：进阶主题，应该什么时候补	10
9 初学者第一轮明确可以不看的文件	11
10 推荐的四周学习安排	11
11 外部参考汇总	12
12 结论	13

1 文档定位

这份文档面向“刚开始系统学习 Linux 内存管理 (mm)”的读者，目标不是覆盖 mm/ 目录下所有子系统，而是提供一条对初学者成本最低、收益最高的阅读路径。

本文的使用方式

- 先按本文给出的顺序建立整体地图，再去读代码。
- 每一块都用三种优先级标记：
 - 必须掌握**：第一轮必须看懂，否则后面的代码会断层。
 - 后续再看**：建议在主路径打通后再补。
 - 暂时跳过**：先不要投入时间，避免刚入门就被支线拖住。
- 这份文档以仓库内源码树为准。外部链接在 2026-04-12 检查过，但在线文档可能比本地 7.0.0-rc7 略有漂移；读实现细节时优先看本地源码。

阅读总原则

- 先弄清“对象和层次”，再看算法细节。
- 先打通主路径，再进入优化路径和异常路径。
- 先选一套架构 fault 路径，推荐只看 x86。
- 不要一上来平铺阅读 mm/*.c；这样最容易迷路。

2 整体路线总表

主题块	先看文档	先看代码	第一轮目标
总览与术语	Documentation/mm/index.rst Documentation/admin-guide/mm/concepts.rst	include/linux/mm_types.h include/linux/mmzone.h include/linux/gfp_types.h	建立 node/zone/folio/VMA/page table/reclaim 的全局地图。
物理内存与页分配	Documentation/mm/physical_memory.rst	mm/page_alloc.c mm/compaction.c	理解 page allocator 在分配、回收、碎片化之间如何联动。

主题块	先看文档	先看代码	第一轮目标
地址空间、页表与缺页	Documentation/mm/process_addrs.rst Documentation/mm/page_tables.rst	mm/mmap.c mm/vma.c mm/memory.c arch/x86/mm/fault.c	打通 mmap -> page fault -> anon/file fault -> COW 主路径。
页缓存与文件映射	Documentation/mm/page_cache.rst	mm/filemap.c mm/readahead.c mm/workingset.c	理解文件读写、mmap 文件页 fault、readahead 和 refault。
回收、交换与OOM	Documentation/admin-guide/mm/concepts.rst	mm/vmscan.c mm/rmap.c mm/swap_state.c mm/swapfile.c mm/oom_kill.c	理解为什么能回收、如何回收、何时会 swap/OOM。
进阶主题	Documentation/mm/transhuge.rst Documentation/mm/multigen_lru.rst Documentation/mm/numa.rst	mm/huge_memory.c mm/khugepaged.c mm/memcontrol.c	在主线走通后再进入 THP、memcg、NUMA、MGLRU。

3 第 0 块：先建立脑图，不要急着看所有子系统

这一步必须掌握的对象

- 必须掌握 物理内存层： node、zone、page frame、folio。
- 必须掌握 虚拟地址空间层： mm_struct、vm_area_struct、page table。
- 必须掌握 文件与缓存层： address_space、page cache、readahead、workingset。
- 必须掌握 压力处理层： reclaim、swap、OOM、compaction。
- 后续再看 控制与策略层： THP、memcg、NUMA policy、MGLRU、DAMON、HMM。

最先应该记住的结构体

- 必须掌握 include/linux/mm_types.h 里的 struct page、struct folio、struct vm_area_struct、struct mm_struct。
- 必须掌握 include/linux/mmzone.h 里的 struct zone、struct per_cpu_pages、struct pglist_data。
- 必须掌握 include/linux/gfp_types.h 里的 gfp_t 和各种 __GFP_* 语义。

现在先不要深挖的内容

- 暂时跳过 DAMON、HMM、secretmem、hwpoison、userfaultfd。
- 暂时跳过 CMA、memory_hotplug、balloon、memory-tiers。
- 暂时跳过 nommu 和大部分 highmem 细节，除非你明确要做 32 位内核学习。

外部参考

- 官方总览：<https://docs.kernel.org/mm/index.html>
- 概念总览：<https://docs.kernel.org/admin-guide/mm/concepts.html>
- 历史技术书目录（适合查章节，不适合直接对号入座当前实现）：<https://www.kernel.org/doc/gorman/html/understand/understand001.html>

4 第 1 块：物理内存、页分配与分配失败后的联动

为什么从这里开始

如果你不知道 page allocator 在做什么，那么后面看到 page fault、page cache、slab、swap 时都会停留在“它又去拿了一页”的表面层。Linux mm 的很多路径最终都会落到 `alloc_pages()` 或其慢路径上。

本块重点知识点

- 必须掌握 node / zone / zonelist。分配不是在“全局内存池”里随便拿，而是按 node、zone 和 fallback 顺序选择。
- 必须掌握 order 与 buddy allocator。理解 order-0、order-*n*、split、merge 和外部碎片化。
- 必须掌握 per-cpu pagesets。绝大多数 order-0 分配并不直接碰全局 buddy。
- 必须掌握 GFP 标志。GFP_KERNEL、__GFP_DIRECT_RECLAIM、__GFP_KSWAPD_RECLAIM、__GFP_HIGH、__GFP_THISNODE 到底影响什么。
- 必须掌握 快路径和慢路径。先成功从 freelist/pcp 拿页；失败后才进入 direct reclaim、compaction、OOM 相关慢路径。
- 后续再看 watermark、boost、reserve。第一轮要知道它们存在并影响回收时机，不必一开始背所有细节。
- 后续再看 migratetype 与 pageblock。它们和 compaction、CMA、碎片化治理强相关。
- 暂时跳过 memblock、sparsemem、vmemmap dedup。这些更适合作为第二轮专题。

第一轮源码入口

- 必须掌握 mm/page_alloc.c:

- `__alloc_pages_noprof()`
- `get_page_from_freelist()`
- `__alloc_pages_slowpath()`
- `rmqueue()`
- `__free_one_page()`
- **必须掌握** `include/linux/mmzone.h`: 先看 `enum zone_type`、`struct zone`、`struct per_cpu_pages`、`struct pglist_data`。
- **必须掌握** `include/linux/gfp_types.h`: 先建立 GFP 语义，不要死记组合宏。
- **后续再看** `mm/compaction.c`: 先知道高阶分配失败后为什么会牵出 `compaction`。

建议阅读顺序

1. `Documentation/mm/physical_memory.rst`
2. `include/linux/mmzone.h`
3. `include/linux/gfp_types.h`
4. `mm/page_alloc.c`
5. 再回来看一次 `page_alloc.c` 里的 `slowpath`

对应外部资料

- 官方文档: https://docs.kernel.org/mm/physical_memory.html
- Mel Gorman, *Describing Physical Memory*: <https://www.kernel.org/doc/gorman/html/understand/understand005.html>
- Mel Gorman, *Physical Page Allocation*: <https://www.kernel.org/doc/gorman/html/understand/understand009.html>
- Linux-MM wiki, *PageAllocation*: <https://linux-mm.org/PageAllocation>

如何使用这些外部资料

- 官方文档用于对照当前对象和术语。
- Gorman 的书适合建立 `buddy`、`zone`、`per-cpu pages` 的基本直觉，但字段和函数名有时代差。
- Linux-MM wiki 适合把页分配器放回设计背景里看，不要直接拿它解释当前代码分支。

5 第 2 块：进程地址空间、VMA、页表与缺页异常

为什么这是整条主线的核心

对新手来说，Linux mm 最重要的闭环不是“某个算法”，而是：

mmap/brk → 建立 VMA → CPU 访问地址 → page fault → `handle_mm_fault()` → 匿名页 / 文件页 / COW / swap fault

只要这条路径走通，后面看 rmap、reclaim、swap、THP 都会容易很多。

本块重点知识点

- **必须掌握** `mm_struct` 和 `vm_area_struct`。一个进程地址空间有哪些 VMA，它们如何被维护。
- **必须掌握** `maple tree`。当前主线不再是老资料里常说的 VMA 红黑树主视角。
- **必须掌握** VMA 锁语义。`mmap_lock`、VMA lock、rmap lock 是不同层次的问题。
- **必须掌握** 五级页表抽象。PGD/P4D/PUD/PMD/PTE 的层次、folding 和 huge mapping。
- **必须掌握** 匿名缺页、文件缺页、写时复制 (COW)。
- **必须掌握** fault 路径的入口。对 x86 来说，从 `arch/x86/mm/fault.c` 进 `handle_mm_fault()`。
- **后续再看** NUMA fault、per-VMA lock 优化、huge fault。第一轮知道它们在主路径里出现即可。
- **后续再看** `mprotect/mremap/userfaultfd/GUP/mmu notifier`。它们很重要，但不应先于基础缺页路径。

第一轮必看源码

- **必须掌握** `include/linux/mm_types.h`: `struct vm_area_struct`、`struct mm_struct`。
- **必须掌握** `mm/mmap.c`: `do_mmap()`。
- **必须掌握** `mm/vma.c`: `mmap_region()`、`vma_merge_new_range()`、`do_vmi_munmap()`。
- **必须掌握** `mm/memory.c`: `handle_mm_fault()`、`__handle_mm_fault()`、`do_anonymous_page()`、`do_wp_page()`、`do_swap_page()`、`finish_fault()`。
- **必须掌握** `arch/x86/mm/fault.c`: `do_user_addr_fault()`，只选一个架构先看。

建议阅读顺序

1. `Documentation/mm/process_addrs.rst`
2. `Documentation/mm/page_tables.rst`
3. `include/linux/mm_types.h`
4. `mm/mmap.c` 与 `mm/vma.c`
5. `arch/x86/mm/fault.c`
6. `mm/memory.c`

第一轮可以先不看的内容

- 暂时跳过 `gup.c` 和 `process_vm_access.c`
- 暂时跳过 `userfaultfd.c`
- 暂时跳过 `mmu_notifier.c`
- 暂时跳过 `mseal.c`、`secretmem.c` 等专项功能

对应外部资料

- 官方文档, *Process Addresses*: https://docs.kernel.org/mm/process_addr.html
- 官方文档, *Page Tables*: https://docs.kernel.org/mm/page_tables.html
- Mel Gorman, *Page Table Management*: <https://www.kernel.org/doc/gorman/html/understand/understand006.html>
- Mel Gorman, *Process Address Space*: <https://www.kernel.org/doc/gorman/html/understand/understand007.html>
- *active_mm* 背景说明 (Linus 在 lore 上的原始解释): <https://lore.kernel.org/lkml/Pine.LNX.4.10.9907301410280.752-100000@penguin.transmeta.com/>

特别提醒 很多旧资料会说“VMA 放在红黑树里”，这对理解历史没问题，但对当前主线不够准确。现在一定要把“mm 内部使用 maple tree 管理 VMA”记牢。

6 第 3 块：页缓存、文件映射与文件页 fault

为什么这一块很早就要看

初学者往往把“缺页”误认为只处理匿名页。实际上文件映射、普通 `read()`、`readahead`、`cache refault`、`workingset` 这些路径，决定了你能不能真正理解 Linux 如何把文件系统和内存管理接起来。

本块重点知识点

- **必须掌握** `page cache` 的对象模型。文件页现在主要以 `folio` 为单位管理。
- **必须掌握** `address_space`。文件页不是凭空存在，它们挂在 `mapping` 上。
- **必须掌握** 文件页 `fault`。`filemap_fault()` 与匿名缺页是两条不同但会汇合的路径。
- **必须掌握** `readahead`。同步/异步 `readahead` 与实际 `cache miss` 路径的关系。
- **必须掌握** `workingset` / `refault`。这是当前 `reclaim` 与 `page cache` 反馈的关键组成部分。
- **后续再看** `truncate`、`writeback`、`dirty throttling`。很重要，但第一轮只需建立接口感。
- **后续再看** `tmpfs/shmem`。它把“匿名”和“文件页”的边界进一步揉在一起，第二轮再细看。

第一轮必看源码

- **必须掌握** mm/filemap.c: filemap_fault()、filemap_read()。
- **必须掌握** mm/readahead.c: page_cache_ra_unbounded()。
- **必须掌握** mm/workingset.c: workingset_eviction()、workingset_refault()、workingset_activation()。
- **后续再看** mm/shmem.c: 理解 tmpfs/shmem 时再细看。

建议阅读顺序

1. Documentation/mm/page_cache.rst
2. include/linux/pagemap.h
3. mm/filemap.c
4. mm/readahead.c
5. mm/workingset.c

这块先不用深挖的内容

- **暂时跳过** mm/page-writeback.c
- **暂时跳过** mm/mapping_dirty_helpers.c
- **暂时跳过** mm/fadvise.c、mm/msync.c

对应外部资料

- 官方文档, *Page Cache*: https://docs.kernel.org/next/mm/page_cache.html
- 官方文档, *Memory Management APIs / Filemap*: <https://docs.kernel.org/core-api/mm-api.html>
- Mel Gorman, *Page Frame Reclamation* 中的 page cache 部分: <https://www.kernel.org/doc/gorman/html/understand/understand013.html>

特别提醒 如果你看的资料大量使用“page”作为页缓存核心单位,不必惊慌。当前主线里很多接口已经 folio 化,但核心问题还是那几个:页在不在缓存里、是不是 uptodate、要不要触发 readahead、refault 能不能说明当前 working set 被挤掉了。

7 第 4 块：回收、反向映射、交换与 OOM

为什么这一块不能拖到最后

一旦分配失败、内存变紧, Linux mm 的主线就会迅速转向 reclaim、rmap、swap、OOM。这部分如果不懂,你就解释不了:

- 为什么某些页能被回收,某些不能;

- 为什么 reclaim 需要知道一个 folio 被哪些页表映射；
- 为什么有时先 direct reclaim，有时唤醒 kswapd；
- 为什么最后会走到 OOM killer。

本块重点知识点

- **必须掌握** reclaimable 与 unreclaimable 的区别。
- **必须掌握** file/anon 两类页在 reclaim 中的不同命运。
- **必须掌握** rmap 的目的。不是“多维护一个映射”而已，而是 reclaim/COW/migration 都要靠它反查。
- **必须掌握** try_to_unmap。页要被回收前，往往先得把页表映射拆掉。
- **必须掌握** kswapd 与 direct reclaim。
- **必须掌握** swap cache 与 swapin readahead。
- **必须掌握** OOM 的触发位置与分工。
- 后续再看 workingset feedback、MGLRU、refault distance。
- 后续再看 zswap/zsmalloc。它们是 swap 的优化层，不是第一层机制。

第一轮必看源码

- **必须掌握** mm/rmap.c:folio_add_new_anon_rmap(),rmap_walk(),try_to_unmap(),folio_referenced()。
- **必须掌握** mm/vmscan.c:try_to_free_pages(),shrink_node(),balance_pgdat(),wakeup_kswapd()。
- **必须掌握** mm/swap_state.c: read_swap_cache_async(), swapin_readahead()。
- **必须掌握** mm/swapfile.c: 先建立 swap_info_struct 和 swap area 的概念。
- **必须掌握** mm/oom_kill.c:out_of_memory(),select_bad_process(),oom_kill_process()。

建议阅读顺序

1. 先重读 Documentation/admin-guide/mm/concepts.rst 里的 reclaim/compaction/OOM 部分
2. mm/rmap.c
3. mm/vmscan.c
4. mm/swap_state.c 与 mm/swapfile.c
5. mm/oom_kill.c

第一轮可以暂缓

- 暂时跳过 `mm/zswap.c`
- 暂时跳过 `mm/zsmalloc.c`
- 暂时跳过 `mm/page_owner.c`
- 暂时跳过 `mm/page_table_check.c`

对应外部资料

- 官方概念文档(reclaim/OOM 总览):<https://docs.kernel.org/admin-guide/mm/concepts.html>
- 官方文档, *Multi-Gen LRU*:https://docs.kernel.org/admin-guide/mm/multigen_lru.html
- Mel Gorman, *Page Frame Reclamation*:<https://www.kernel.org/doc/gorman/html/understand/understand013.html>
- Mel Gorman, *Swap Management*:<https://www.kernel.org/doc/gorman/html/understand/understand014.html>
- Linux-MM wiki, *PageReplacementDesign*: <https://linux-mm.org/PageReplacementDesign>

如何读这块 第一轮不要试图把 `mm/vmscan.c` 每个分支一次性吃掉。先抓四个问题：

1. 谁在发起 reclaim?
2. 这次优先回收 file 还是 anon?
3. 页在回收前为什么需要 rmap/try-to-unmap?
4. reclaim 失败后什么时候会走到 OOM?

8 第 5 块：进阶主题，应该什么时候补

这一块不是“不重要”，而是“不应抢跑”

THP、memcg、NUMA、MGLRU、DAMON、HMM 都是现代 Linux mm 的关键主题，但它们都建立在前面几块已经打通的前提之上。对初学者来说，最常见的问题不是“这些特性太难”，而是“在主线还没通的时候太早进去”。

推荐顺序

1. **必须掌握** THP：先看 PMD-sized huge page fault、collapse、khugepaged。
2. **后续再看** MGLRU：在你已经理解传统 reclaim、refault、workingset 之后再进入。
3. **后续再看** memcg：在你已经知道全局回收是怎么回事后，再看 cgroup 维度的回收。
4. **后续再看** NUMA：先掌握 node/zone，再进 mempolicy 和自动 NUMA balancing。

5. **暂时跳过** DAMON/HMM/memory tiers: 这些更像专题, 不适合第一轮。

推荐入口

- THP: Documentation/mm/transhuge.rst, Documentation/admin-guide/mm/transhuge.rst, mm/huge_memory.c, mm/khugepaged.c
- MGLRU: Documentation/mm/multigen_lru.rst, Documentation/admin-guide/mm/multigen_lru.rst
- memcg: mm/memcontrol.c
- NUMA: Documentation/mm/numa.rst, Documentation/admin-guide/mm/numa_memory_policy.rst, mm/mempolicy.c

对应外部资料

- THP 官方文档: <https://docs.kernel.org/admin-guide/mm/transhuge.html>
- MGLRU 官方文档: https://docs.kernel.org/admin-guide/mm/multigen_lru.html
- 进程页表检查工具: <https://docs.kernel.org/admin-guide/mm/pagemap.html>

9 初学者第一轮明确可以不看的文件

下面这些不是“不重要”，而是“对第一轮收益不高”：

- **暂时跳过** mm/damon/
- **暂时跳过** mm/hmm.c
- **暂时跳过** mm/userfaultfd.c
- **暂时跳过** mm/secretmem.c
- **暂时跳过** mm/hwpoison*
- **暂时跳过** mm/cma*
- **暂时跳过** mm/memory_hotplug.c
- **暂时跳过** mm/page_owner.c、mm/debug*.c、mm/*test*.c
- **暂时跳过** 同时比较 x86/arm64/riscv 三套 fault 路径

10 推荐的四周学习安排

第 1 周：地图与基础对象

- 目标：建立全局术语地图。

- 阅读: `Documentation/admin-guide/mm/concepts.rst`、`Documentation/mm/physical_memory.rst`、`Documentation/mm/page_tables.rst`、`Documentation/mm/process_addrs.rst`。
- 代码: `include/linux/mm_types.h`、`include/linux/mmzone.h`、`include/linux/gfp_types.h`。

第 2 周：分配与地址空间

- 目标：理解 `alloc_pages()` 和 `mmap()` 是什么。
- 代码： `mm/page_alloc.c`、`mm/mmap.c`、`mm/vma.c`。
- 产出：能画出一个 VMA 被建立出来的大致流程。

第 3 周：缺页与文件映射

- 目标：打通 `page fault` 主路径。
- 代码： `arch/x86/mm/fault.c`、`mm/memory.c`、`mm/filemap.c`、`mm/readahead.c`。
- 产出：能解释 `anon fault`、`file fault`、`COW` 的区别。

第 4 周：回收与交换

- 目标：理解内存压力下系统为什么这样行动。
- 代码： `mm/rmap.c`、`mm/vmscan.c`、`mm/swap\_state.c`、`mm/swapfile.c`、`mm/oom\_kill.c`。
- 产出：能解释 `direct reclaim`、`kswapd`、`swap`、`OOM` 之间的关系。

11 外部参考汇总

官方与当前文档

- <https://docs.kernel.org/mm/index.html>
- <https://docs.kernel.org/admin-guide/mm/concepts.html>
- https://docs.kernel.org/mm/physical_memory.html
- https://docs.kernel.org/mm/page_tables.html
- https://docs.kernel.org/mm/process_addrs.html
- https://docs.kernel.org/next/mm/page_cache.html
- <https://docs.kernel.org/admin-guide/mm/transhuge.html>
- https://docs.kernel.org/admin-guide/mm/multigen_lru.html
- <https://docs.kernel.org/admin-guide/mm/pagemap.html>

历史技术文档与长期参考

- Mel Gorman, *Understanding the Linux Virtual Memory Manager*: <https://www.kernel.org/doc/gorman/html/understand/understand001.html>
- *Describing Physical Memory*: <https://www.kernel.org/doc/gorman/html/understand/understand005.html>
- *Page Table Management*: <https://www.kernel.org/doc/gorman/html/understand/understand006.html>
- *Process Address Space*: <https://www.kernel.org/doc/gorman/html/understand/understand007.html>
- *Physical Page Allocation*: <https://www.kernel.org/doc/gorman/html/understand/understand009.html>
- *Page Frame Reclamation*: <https://www.kernel.org/doc/gorman/html/understand/understand013.html>
- *Swap Management*: <https://www.kernel.org/doc/gorman/html/understand/understand014.html>
- Linux-MM wiki, *PageAllocation*: <https://linux-mm.org/PageAllocation>
- Linux-MM wiki, *PageReplacementDesign*: <https://linux-mm.org/PageReplacementDesign>

如何区分这些资料的用途

- 官方文档：适合对照当前主线术语和对象。
- Gorman：适合理解原理和主线结构，不适合直接对照今天的字段名、锁和具体函数。
- Linux-MM wiki：适合理解设计动机和历史背景，不适合当作当前实现说明书。

12 结论

对初学者来说，Linux mm 最重要的不是一上来知道所有子系统，而是先打通这五件事：

1. 物理内存如何被组织；
2. 地址空间如何被描述；
3. 缺页如何被处理；
4. 文件页如何进入和离开 page cache；
5. 内存不够时 reclaim/swap/OOM 如何联动。

只要这五块走通，再进入 THP、MGLRU、memcg、NUMA，就会是“在现有地图上扩展”，而不是“重新学一套新系统”。